LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 2 – REVIEW PENGENALAN PEMROGRAMAN

NAMA : Wahid Satrio Aji

NIM : 109082500005

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

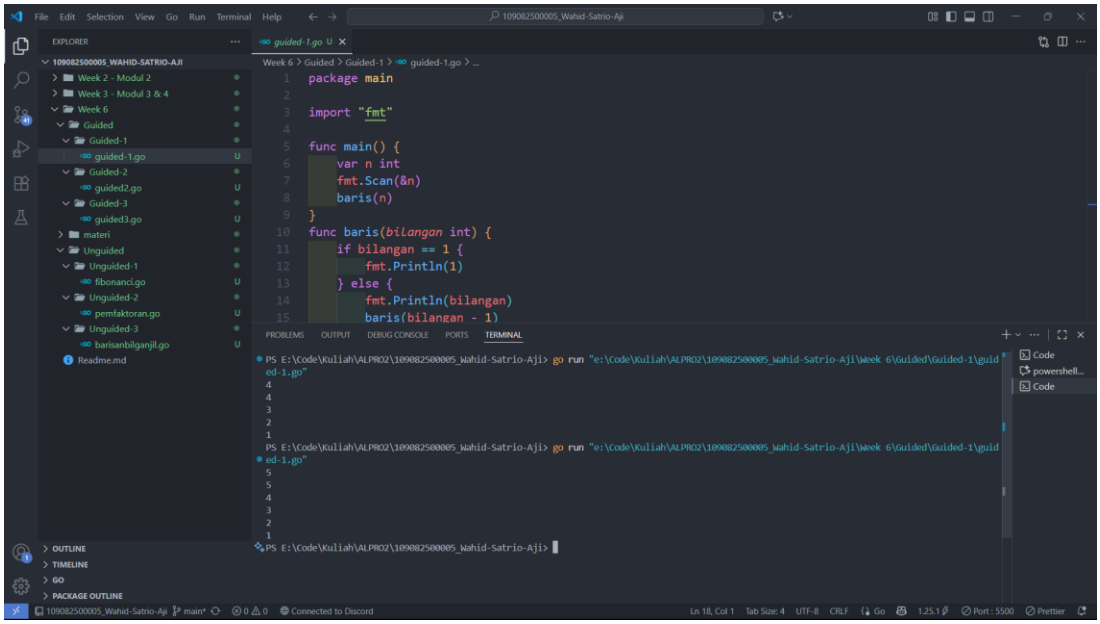
Rekursi atau *recursion* merupakan sebuah teknik dalam algoritma pemrograman di mana sebuah fungsi atau prosedur memanggil dirinya sendiri untuk menyelesaikan suatu masalah. Konsep ini bekerja dengan cara membagi masalah besar menjadi sub-masalah yang lebih kecil dan sejenis hingga mencapai titik yang paling sederhana. Setiap subprogram rekursif wajib memiliki dua komponen utama: kondisi dasar (*base case*) dan langkah rekursif. Kondisi dasar berfungsi sebagai titik henti agar program tidak terjebak dalam perulangan tak terbatas (*infinite loop*), sementara langkah rekursif adalah bagian di mana fungsi memanggil kembali dirinya dengan argumen yang nilainya semakin mendekati kondisi dasar tersebut. Penerapan rekursi sangat efektif untuk menangani masalah yang memiliki pola berulang atau struktur hierarkis, sehingga membuat penulisan kode menjadi lebih ringkas dan logis dibandingkan menggunakan perulangan iteratif biasa.

B. Guided

1. Membuat baris bilangan dari n hingga 1

```
package main

import "fmt"

func main() {
	var n int
	fmt.Scan(&n)
	baris(n)
}
func baris(bilangan int) {
	if bilangan == 1 {
		fmt.Println(1)
	} else {
		fmt.Println(bilangan)
		baris(bilangan - 1)
	}
}
```

Output :



Penjelasan :

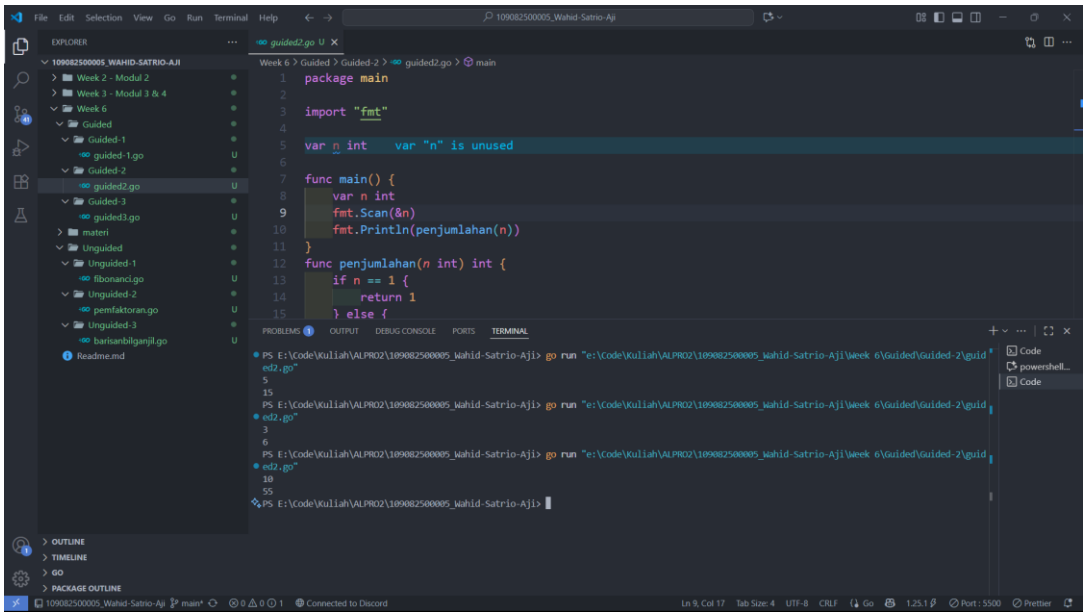
Program ini menggunakan prosedur rekursif bernama baris untuk menampilkan urutan angka secara menurun mulai dari nilai n yang dimasukkan pengguna hingga angka 1. Logika yang digunakan adalah dengan mengecek kondisi dasar; jika angka sudah mencapai 1, maka program akan mencetak angka tersebut dan berhenti. Jika belum, program akan mencetak angka saat ini dan memanggil kembali fungsi baris dengan parameter nilai yang telah dikurangi satu n-1. Hal ini memungkinkan terciptanya barisan bilangan yang menurun secara otomatis tanpa menggunakan struktur perulangan *for* atau *while*.

2. Menghitung hasil penjumlahan 1 hingga n

```
package main

import "fmt"

var n int

func main() {
      var n int
      fmt.Scan(&n)
      fmt.Println(penjumlahan(n))
}
func penjumlahan(n int) int {
      if n == 1 {
            return 1
```

```
        } else {
                return n + penjumlahan(n-1)
        }
}
```

Output :



Penjelasan :

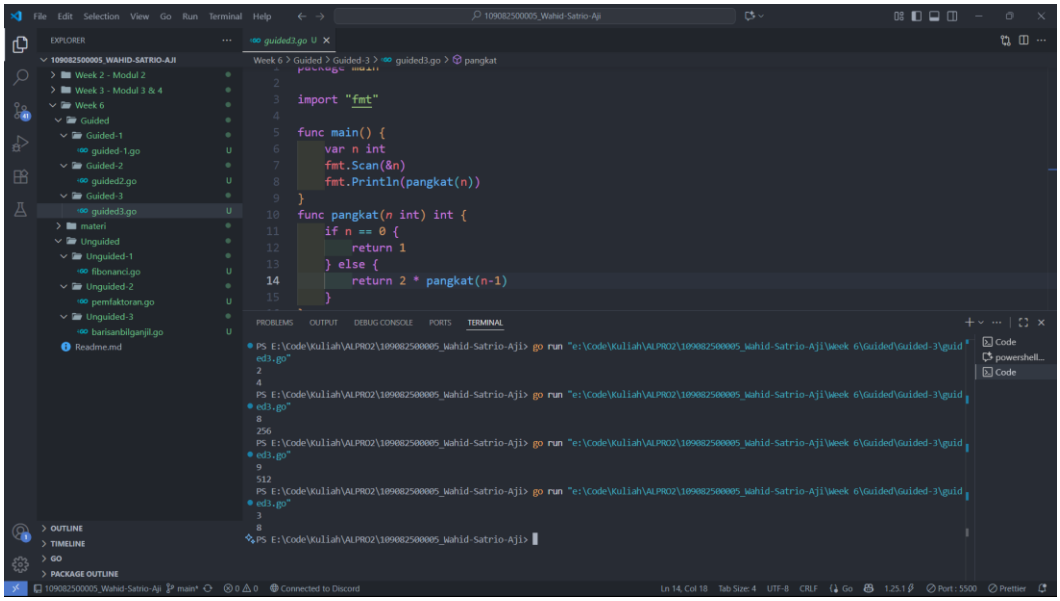
Kode program ini dirancang untuk menghitung total akumulasi bilangan dari 1 sampai n dengan pendekatan rekursi. Fungsi penjumlahan menerima satu parameter masukan dan akan terus memanggil dirinya sendiri dengan nilai n-1 sambil menambahkan nilai n tersebut ke hasil pemanggilan berikutnya. Proses ini akan terus berulang hingga menyentuh kondisi dasar yaitu n == 1, di mana fungsi akan mengembalikan nilai 1. Hasil akhir dari rangkaian penjumlahan tersebut kemudian dikembalikan ke program utama untuk ditampilkan sebagai total nilai keseluruhan.

3. Mencari dua pangkat n atau 2 pangkat n

```
package main

import "fmt"

func main() {
        var n int
        fmt.Scan(&n)
        fmt.Println(pangkat(n))
}
```

```
func pangkat(n int) int {
	if n == 0 {
		return 1
	} else {
		return 2 * pangkat(n-1)
	}
}
```

Output :



Penjelasan :

Program ini mengimplementasikan perhitungan eksponensial, khususnya perpangkatan berbasis dua, melalui fungsi rekursif pangkat. Cara kerjanya adalah dengan mengalikan angka 2 secara berulang sebanyak n kali. Fungsi ini memiliki kondisi dasar di mana jika nilai pangkat adalah 0, maka ia akan mengembalikan nilai 1 sesuai dengan aturan matematika. Selama nilai pangkat belum mencapai 0, fungsi akan terus melakukan perkalian rekursif dengan rumus 2 * pangkat(n-1) hingga seluruh operasi perkalian selesai dan hasil akhir didapatkan.

C. Unguided

1. Soal Unguided 1

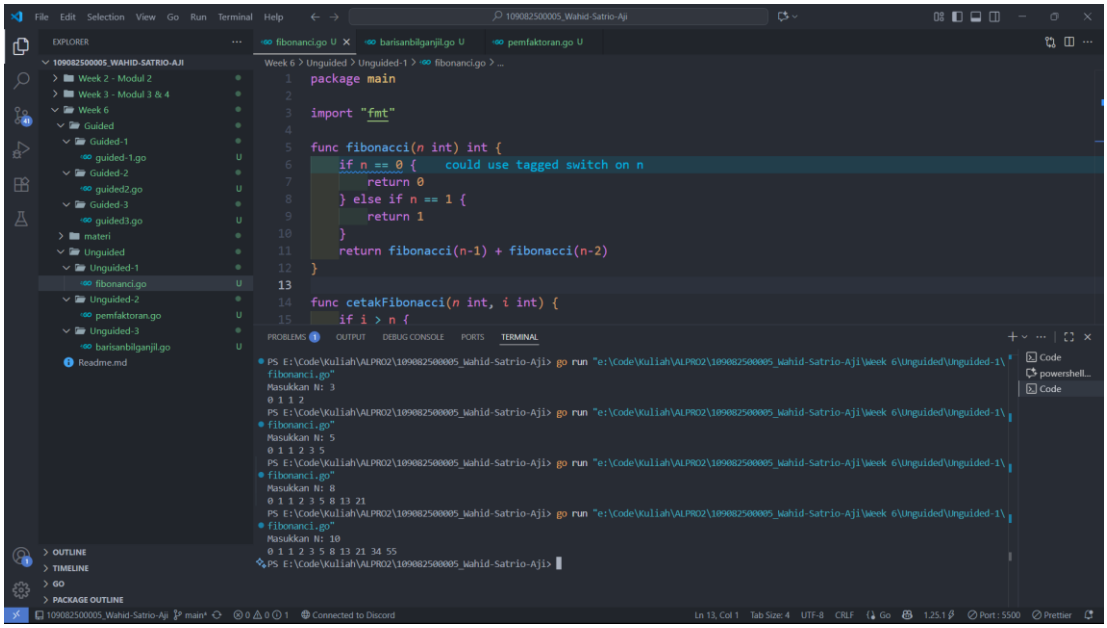
1) Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $S_n = S_{n-1} + S_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

n	0	1	2	3	4	5	6	7	8	9	10
S_n	0	1	1	2	3	5	8	13	21	34	55

Jawaban :

```
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func cetakFibonacci(n int, i int) {
    if i > n {
        return
    }
    fmt.Print(fibonacci(i), " ")
    cetakFibonacci(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)
    cetakFibonacci(n, 0)
    fmt.Println()
}
```

Output :



Penjelasan :

Kode ini bertujuan untuk menghitung dan menampilkan deret Fibonacci hingga suku tertentu. Program menggunakan fungsi rekursif fibonanci yang bekerja dengan menjumlahkan dua suku sebelumnya untuk mendapatkan suku saat ini. Terdapat dua kondisi dasar, yaitu ketika n bernilai 0 atau 1, fungsi akan langsung mengembalikan nilai tersebut. Pada bagian langkah rekursif, fungsi memanggil dirinya sendiri dua kali secara bersamaan, yaitu untuk suku n-1 dan suku n-2. Program utama kemudian menggunakan perulangan untuk memanggil fungsi ini secara bertahap sehingga deret angka dapat tercetak secara berurutan mulai dari suku awal.

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

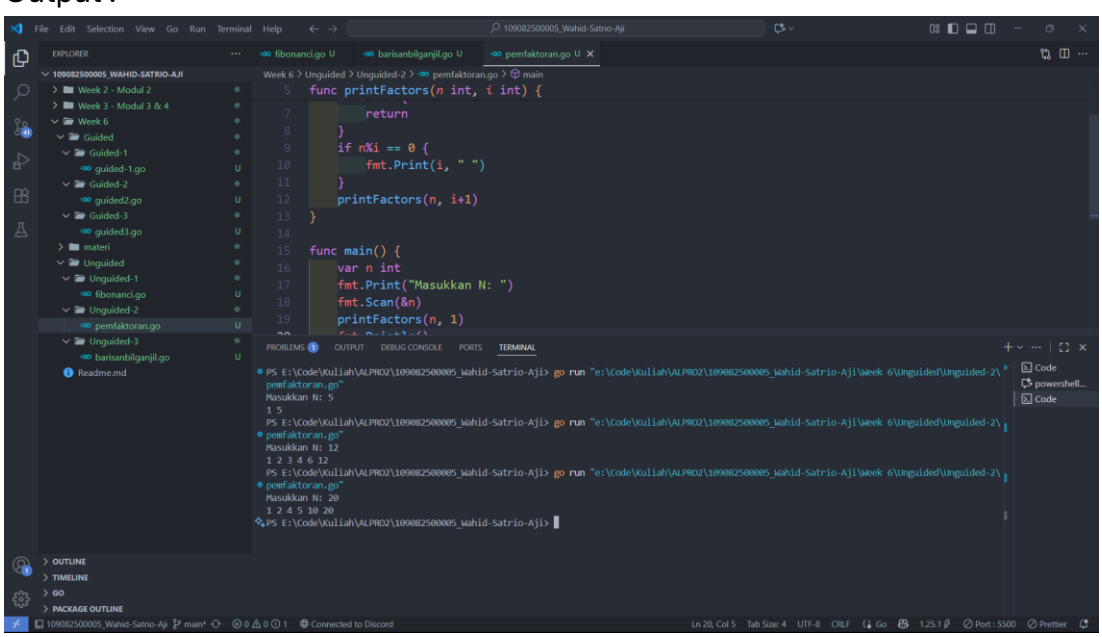
Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	5	1 5

Jawaban :

```go
package main

import "fmt"

func printFactors(n int, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    printFactors(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)
    printFactors(n, 1)
    fmt.Println()
}
```

Output :



Penjelasan :

Program ini dirancang untuk mencari semua bilangan pembagi (faktor) dari suatu angka tertentu yang dimasukkan oleh pengguna. Prosedur rekursif faktor menerima dua argumen: angka target (n) dan angka pembagi (i) yang dimulai dari 1. Program melakukan evaluasi menggunakan operator modulo untuk memeriksa apakah n habis dibagi oleh i. Jika sisa baginya adalah nol, maka i merupakan faktor dan akan dicetak ke layar. Setelah itu, prosedur memanggil dirinya sendiri dengan menambahkan nilai i sebesar 1. Proses ini akan terus berlanjut hingga nilai i melebihi angka target n, yang menjadi batas akhir rekursi.

3. Soal Unguided 3

5) Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Contoh masukan dan keluaran:

Fakultas Informatika
School of Computing
Telkom University
informatics lab

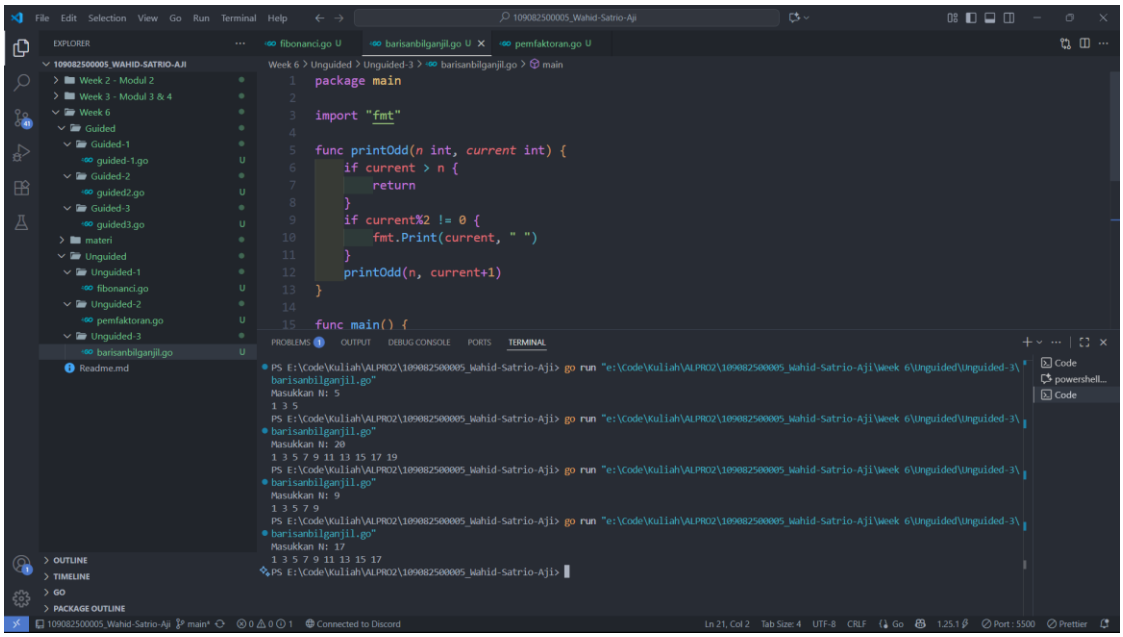
No	Masukan	Keluaran
1	5	1 3 5
2	20	1 3 5 7 9 11 13 15 17 19

Jawaban :

```
package main

import "fmt"

func printOdd(n int, current int) {
    if current > n {
        return
    }
    if current%2 != 0 {
        fmt.Print(current, " ")
    }
    printOdd(n, current+1)
}

func main() {
    var n int
    fmt.Print("Masukkan N: ")
    fmt.Scan(&n)
    printOdd(n, 1)
```

```
            fmt.Println()
        }
```

Output :



Penjelasan :

Kode program ini berfungsi untuk menampilkan seluruh bilangan ganjil yang terdapat dalam rentang antara 1 hingga batas maksimal n. Prosedur rekursif cetakGanjil bekerja dengan memeriksa setiap angka i menggunakan logika sisa bagi; jika angka tersebut tidak habis dibagi dua, maka angka tersebut dikategorikan sebagai bilangan ganjil dan ditampilkan. Setelah melakukan pengecekan, prosedur memanggil dirinya sendiri untuk mengevaluasi angka berikutnya ($i+1$). Rekursi akan berhenti secara otomatis ketika angka yang diperiksa telah melampaui batas nilai n yang ditentukan oleh pengguna, sehingga menghasilkan daftar bilangan ganjil yang sistematis.

D. Kesimpulan

Berdasarkan praktikum yang sudah terjadi, dapat disimpulkan bahwa rekursi merupakan metode pemecahan masalah yang sangat kuat dalam pemrograman karena mampu menyederhanakan algoritma yang memiliki sifat repetitif. Penggunaan rekursi memungkinkan penulisan kode yang lebih modular dan elegan

dibandingkan dengan metode iteratif konvensional. Namun, keberhasilan sebuah program rekursif sangat bergantung pada penentuan kondisi dasar yang tepat guna mencegah terjadinya penggunaan memori yang berlebihan atau kesalahan eksekusi. Melalui latihan pada modul ini, terlihat bahwa rekursi sangat efektif diterapkan pada berbagai persoalan matematika seperti deret Fibonacci, faktorisasi, maupun pengolahan barisan bilangan secara efisien.